



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

严谨 · 勤奋 · 求实 · 创新

数字系统设计

14-端到端部署、处理器级并行 (ALP)

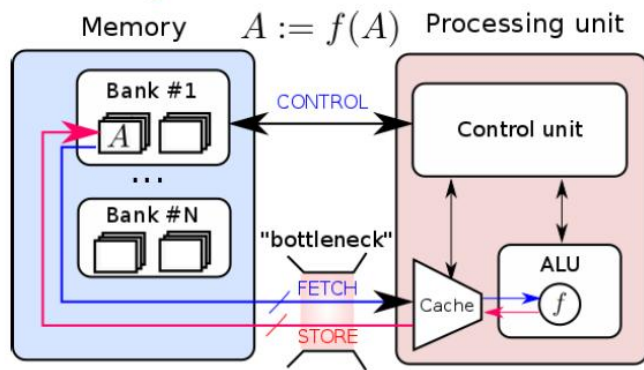
贾弘洋 & 刘勇攀

2025/05/25

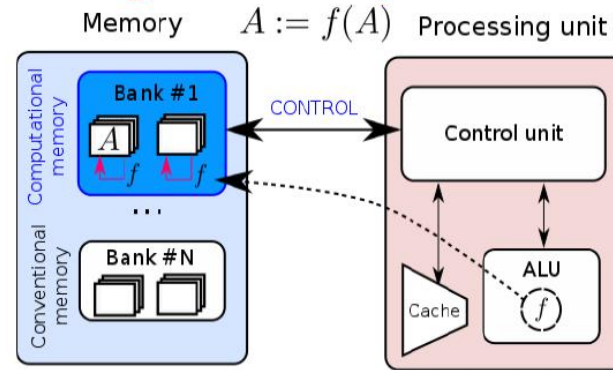
- Introduction: **CIM to break the memory wall**
- Memory devices and computational primitives
 - Charge-based memory devices: **SRAM, DRAM, Flash**
 - Resistance-based memory devices: **ReRAM, PCM, STT-MRAM**
- SRAM-based CIM: from device to architecture
 - 3 SRAM CIM primitives: **current, charge, digital**
 - Toward efficient MVM mapping to CIM array: **OS, matrix mapping**
 - Sparsity in CIM: **structural sparsity for CIM**
 - Challenges and Trade-offs in CIM: **CIM macro portion, accuracy, etc.**

/ Compute in Memory (CIM)

Processing unit & Conventional memory

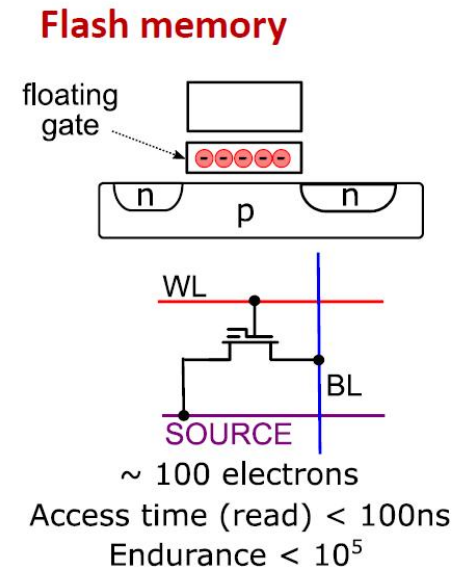
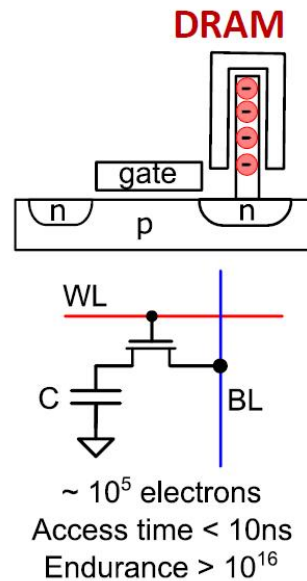
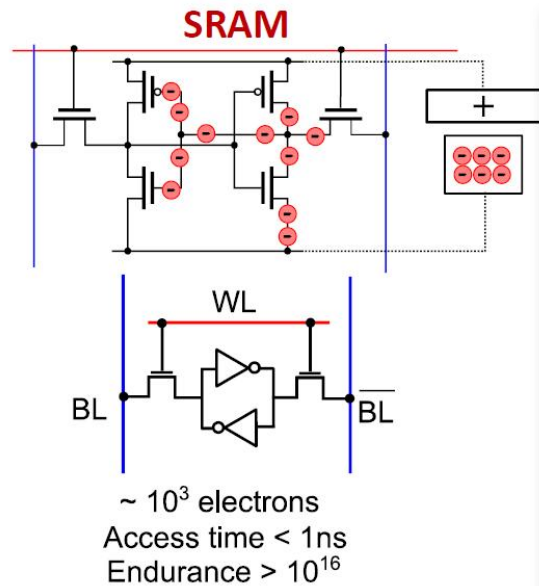


Processing unit & Computational memory



- Perform “certain” computational tasks **in place in memory**
- Achieved by exploiting **the physical attributes of the memory devices**, their **array level organization**, the **peripheral circuitry** as well as the **control logic**
- At **no point during computation**, the memory content is read back and **processed at the granularity of a single memory element**

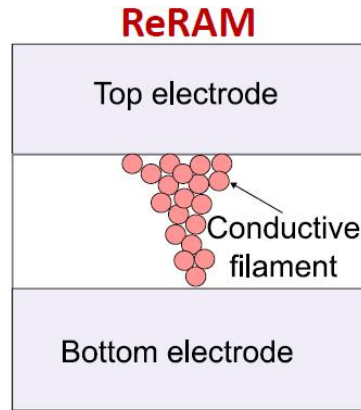
/ Charge-Based Memory Devices



- **SRAM:** Two CMOS inverters connected back to back. The charge is confined within the barriers formed by FET channels and by gate insulators
- **DRAM:** Capacitor connected in series to a FET
- **Flash:** The storage node is coupled to the gate of a FET

*Emerging nanoelectronics
devices, John Wiley & Sons (2015)*

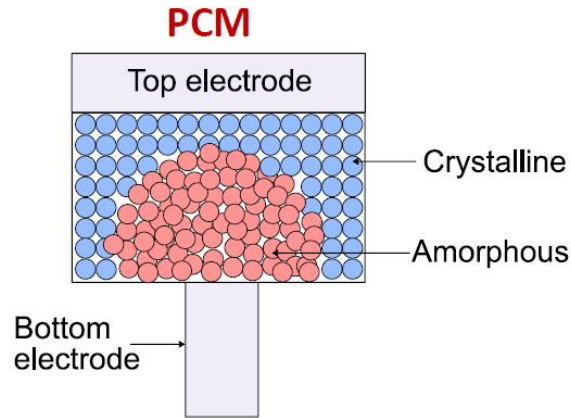
/Resistance-Based Memory Devices



Resistance range = 10^3 - 10^7

Access time (write) = 10ns - 100ns

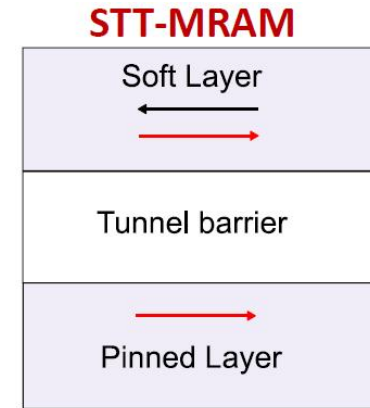
Endurance = 10^6 - 10^9



Resistance range = 10^4 - 10^7

Access time (write) ~ 100ns

Endurance = 10^6 - 10^9



Resistance range = 10^3 - 10^4

Access time (write) < 10ns

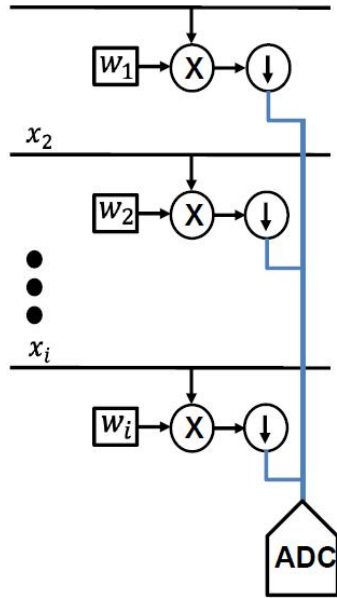
Endurance > 10^{14}

- **ReRAM:** Migration of defects such as oxygen vacancies or metallic ions
- **PCM:** Joule-heating induced reversible phase transition
- **STT-MRAM:** Magnetic polarization of a free layer with respect to a pinned layer
- Resistance-based memory devices also referred to as **memristive devices**

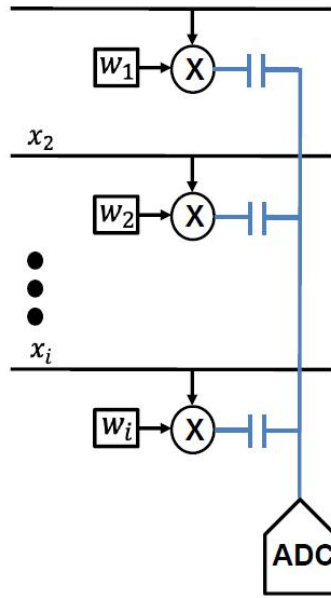
Wong and Salahuddin, Nature Nanotechnology (2015)

Three Types of SRAM-based CIM

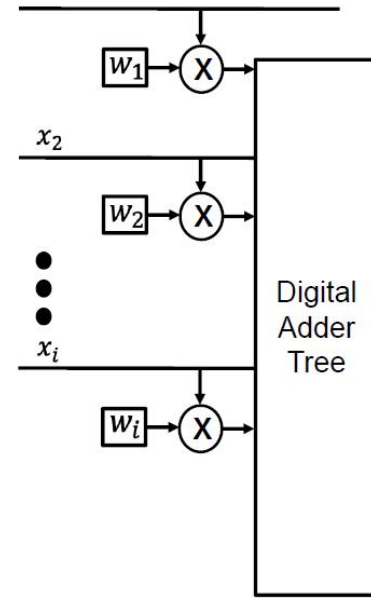
Classified by: Result generation format on each single CIM cell



Current-based CIM
(Less transistors)

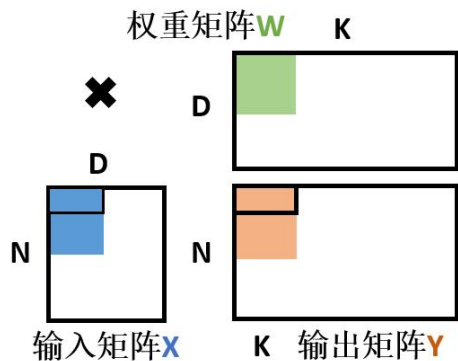


Charge-based CIM
(High parallelism)



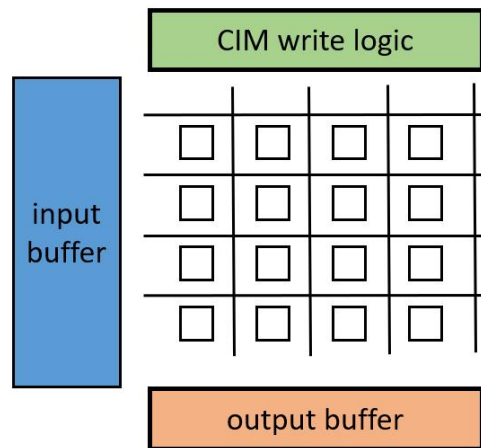
All-digital CIM
(No accuracy loss)

Q4: What if the output buffer size $> 1 \times K_0$? A: Stores > 1 striped rows of output activations. 这里按列存储，考虑 CIM size。



要求: $N_1 \times N_0 = N$ (loop bound)

```
for (k1=0; k1<K1; k1++)
  for (d1=0; d1<D1; d1++)
    UpdateCIM( $W[d1 \times D_0 : (d1+1) \times D_0, k1 \times K_0 : (k1+1) \times K_0]$ ) //WB>=D0*K0
    for (n1=0; n1<N1; n2++)
      mvin( $X[n1 \times N_0 : (n1+1) \times N_0, d1 \times D_0 : (d1+1) \times D_0]$ ) //IB>=1*D0
      if (d1==0)  $Y[n1 \times N_0 : (n1+1) \times N_0, k1 \times K_0 : (k1+1) \times K_0] = 0;$ 
      else mvin( $Y[n1 \times N_0 : (n1+1) \times N_0, k1 \times K_0 : (k1+1) \times K_0]$ ) //OB>=1*K0
      for (n0=0; n0<N0; n0++)
        spatial_for (k0=0; k0<K0; k0++) //column
          spatial_for (d0=0; d0<D0; d0++) //cell
             $Y[n1 \times N_0 + n0, k1 \times K_0 + k0] +=$ 
               $X[n1 \times N_0 + n0, d1 \times D_0 + d0] \times W[d1 \times D_0 + d0, k1 \times K_0 + k0];$ 
      mvout( $Y[n1 \times N_0 : (n1+1) \times N_0, k1 \times K_0 : (k1+1) \times K_0]$ )
```



/Outline

- **End-to-End Deployment**
 - **AI Processing Pipeline**
 - **Practical Constraints:**
Thermal, Performance Variability, Interference
- **Accelerator-Level Parallelism (ALP)**
 - **X-level Parallelism**
 - **ALP in Mobile SoC**
 - **Open Challenges**

/ Holistic View of the ML Processing Pipeline

- End-to-end ML pipeline is more than the ML algorithms.
 - ML algorithms also rely on pre- and post-processing code typically executed on CPUs
- For example:
 - “Data processing machines, aka ‘readers’, read the data from storage, process and condense them, and then send to the training machines.”
 - “Training machines, aka ‘trainers’, solely focus on executing the training options rapidly and efficiently. ”

*Applied Machine Learning at Facebook: A Datacenter Infrastructure Perspective,
HPCA'2018*



/Data Infeed for TPUs

- Data infeed: the process of preparing and moving input data to the TPU board.

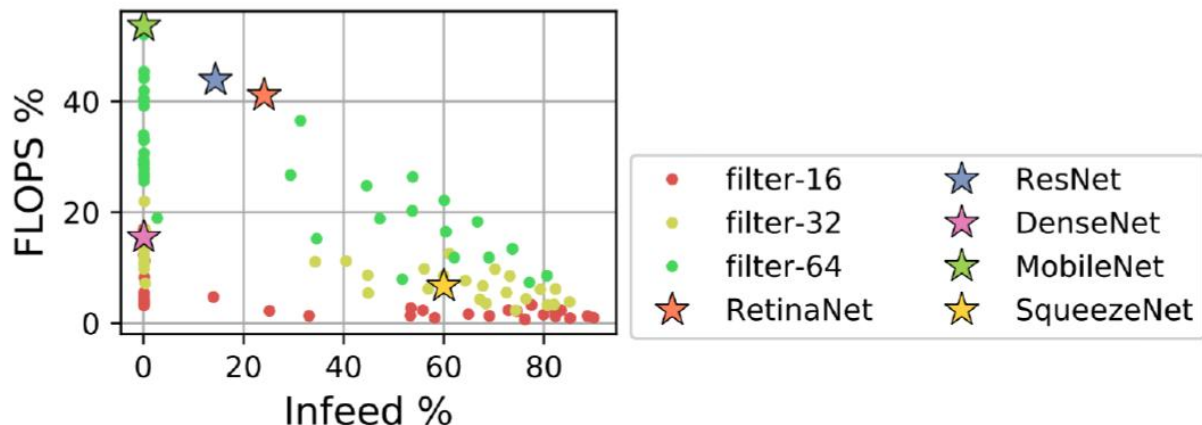
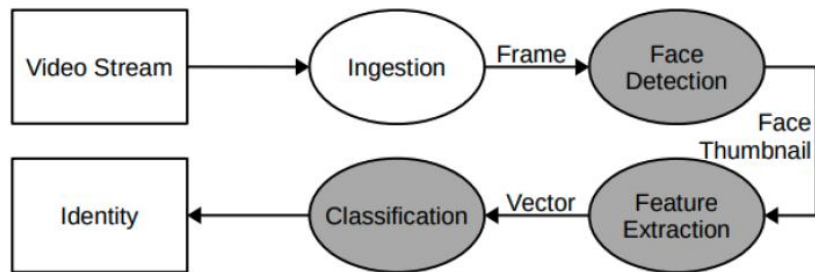
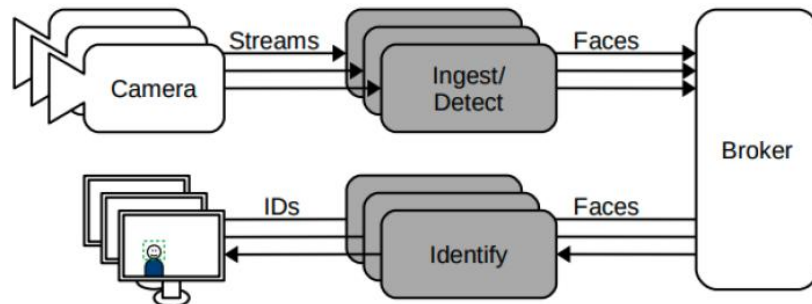


Figure 5: The FLOPS utilizations and data infeed percentages of ParaDnn (dots) and real-world (stars) CNNs.

/ Example: A Face Recognition Pipeline



(a) Algorithmic flow of *Face Recognition*. A video stream enters *ingestion* for separation into individual frames. *Face detection* finds any faces within a frame and produces a thumbnail for each. *Feature extraction* generates identifying features for each face. Finally, *classification* finds a nearest match to known faces to produce an identity.

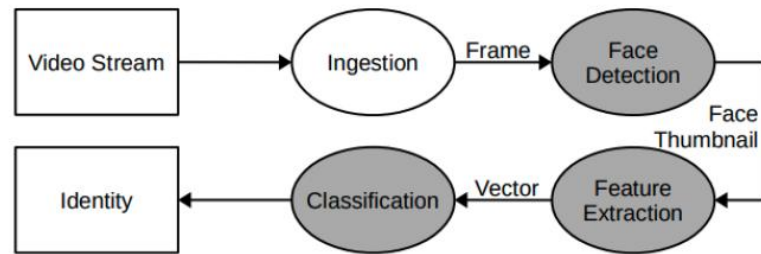


(b) Data center deployment of *Face Recognition*. Algorithms run as standalone processes in lightweight Docker containers deployed on separate nodes throughout the data center. *Ingestion* and *face detection* both run in the *ingest/detect* container while *feature extraction* and *classification* are combined in *identify*. Communication between steps within a container happens internally while communication between containers is coordinated through Apache Kafka *brokers*.

Figure 1: From the conceptual operation of *Face Recognition* to a deployment-ready implementation. Shaded blocks show AI stages. The straightforward application steps are wrapped in containers for deployment in a data center, requiring inputs, outputs, and communication coordination.

/ Example: A Face Recognition Pipeline

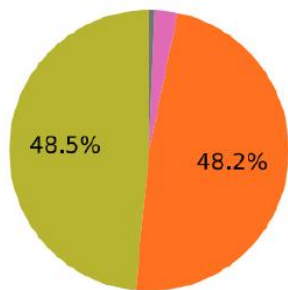
- **Ingestion:** a pre-processing stage that ingests a video stream and parses it into individual frames.
- **Face detection (ML):** Detect any faces within a frame. It determines bounding boxes for and produces a thumbnail of each face in a frame.
- **Feature extraction (ML):** Produce a 128-byte vector of essential features that describe each face.
- **Classification (ML):** compares the feature vector against known face vectors to find the best match.



(a) Algorithmic flow of Face Recognition. A video stream enters ingestion for separation into individual frames. Face detection finds any faces within a frame and produces a thumbnail for each. Feature extraction generates identifying features for each face. Finally, classification finds a nearest match to known faces to produce an identity.

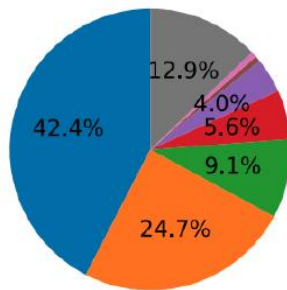
/ Example: A Face Recognition Pipeline

- CPU time breakdown for different components.
- With Amdahl's law, 8X AI speedup $\rightarrow$ 1.59X detection speedup.



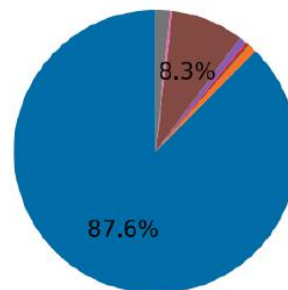
■ Frame Extraction
■ Frame Resize
■ Logging
■ Other

(a) Ingestion.



■ AI
■ Face Resize
■ Networking
■ Other Tensorflow
■ Numpy
■ Kafka
■ Logging
■ Other

(b) Detection.



■ AI
■ Image Mgmt
■ Networking
■ Other Tensorflow
■ Numpy
■ Kafka
■ Logging
■ Other

(c) Identification.

*Missing the Forest for the Trees:
End-to-End AI Application
Performance in Edge Data
Centers, HPCA'2020*

/Outline

- **End-to-End Deployment**
 - **AI Processing Pipeline**
 - **Practical Constraints:**
Thermal, Performance Variability, Interference
- **Accelerator-Level Parallelism (ALP)**
 - **X-level Parallelism**
 - **ALP in Mobile SoC**
 - **Open Challenges**

/ Practical Constraints in Deploying DL

- DL hardware design may seem easy after this course 😊
- Deploying DL hardware at scale requires in-depth understanding of the eco-system.
- CPU is still widely used for DL execution in a range of scenarios.

Service	Resource	Training Frequency	Training Duration
News Feed	Dual-Socket CPUs	Daily	Many Hours
Facer	GPUs + Single-Socket CPUs	Every N Photos	Few Seconds
Lumos	GPUs	Multi-Monthly	Many Hours
Search	Vertical Dependent	Hourly	Few Hours
Language Translation	GPUs	Weekly	Days
Sigma	Dual-Socket CPUs	Sub-Daily	Few Hours
Speech Recognition	GPUs	Weekly	Many Hours

TABLE II

FREQUENCY, DURATION, AND RESOURCES USED BY OFFLINE TRAINING FOR VARIOUS WORKLOADS

Services	Relative Capacity	Compute	Memory
News Feed	100X	Dual-Socket CPU	High
Facer	10X	Single-Socket CPU	Low
Lumos	10X	Single-Socket CPU	Low
Search	10X	Dual-Socket CPU	High
Language Translation	1X	Dual-Socket CPU	High
Sigma	1X	Dual-Socket CPU	High
Speech Recognition	1X	Dual-Socket CPU	High

TABLE III

RESOURCE REQUIREMENTS OF ONLINE INFERENCE WORKLOADS.

/ Practical Constraints in Deploying DL

- DL hardware design may seem easy after this course 😊
- Deploying DL hardware at scale requires in-depth understanding of the eco-system.
- CPU is still widely used for DL execution in a range of scenarios.

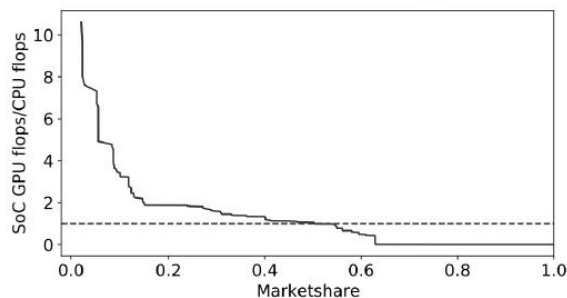
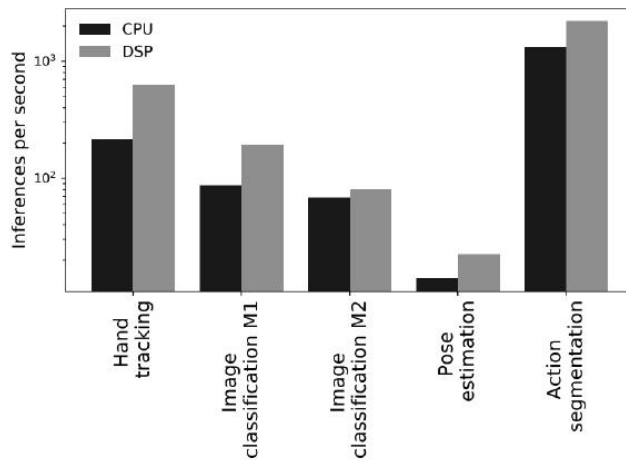


Figure 4: The theoretical peak performance difference between mobile CPUs and GPUs is narrow. In a median Android device, GPU provides only as much performance as its CPU. Only 11% of the smartphones have a GPU that is 3 times more performant than its CPU.



*Machine Learning at Facebook:
Understanding Inference at the
Edge, HPCA'2019*

/Thermal Constraints

- Running multiple DNN inferences for Oculus Virtual Reality.
- High performance requirements:
 - Real-time w/ 30-60 frames per second.
 - Multiple cameras to cover a wide field of view
 - On-device processing
- E.g., Qualcomm Snapdragon SoC, with
 - The CPU model utilizes a big.LITTLE core cluster with 4 Cortex-A73 and 4 Cortex-A53 and a Hexagon DSP.

DNN features	DNN models	MACs	Weights
Hand Tracking	U-Net [29]	10x	1x
Image Model-1	GoogLeNet [30]	100x	1x
Image Model-2	ShuffleNet [27]	10x	2x
Pose Estimation	Mask-RCNN [28]	100x	4x
Action Segmentation	TCN [31]	1x	1.5x

*Machine Learning at Facebook:
Understanding Inference at the
Edge, HPCA'2019*

/ Thermal Constraints

- The thermal throttling has a significant effect on performance, degrading the FPS performance to 10 frames-per-second.
- Despite higher performance, lower power consumption and operating temperature, the DSP implementation comes with significantly higher programming overhead.

*Machine Learning at Facebook:
Understanding Inference at the
Edge, HPCA'2019*

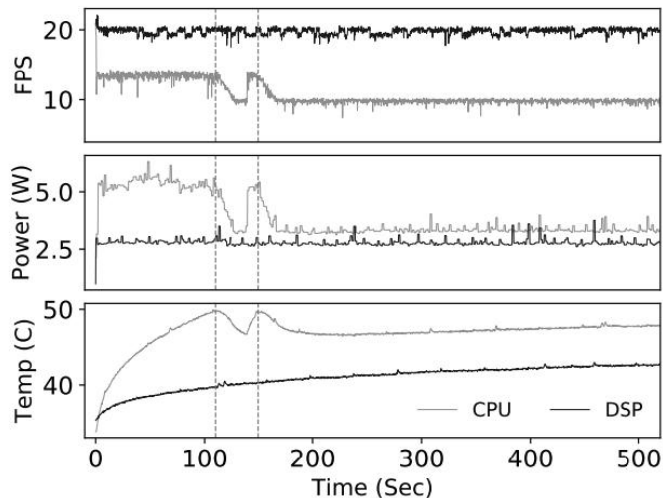


Figure 9: Inference frames-per-second performance, power, and temperature comparison for a vision model for Oculus. Thermal throttling (marked with dotted lines) prevents CPU from operating at an acceptable FPS performance level.

/Performance Variability

- Challenging to make guarantees on quality of service.
- Likely due to system activities in deployed smartphones and the environment the smartphones are in.

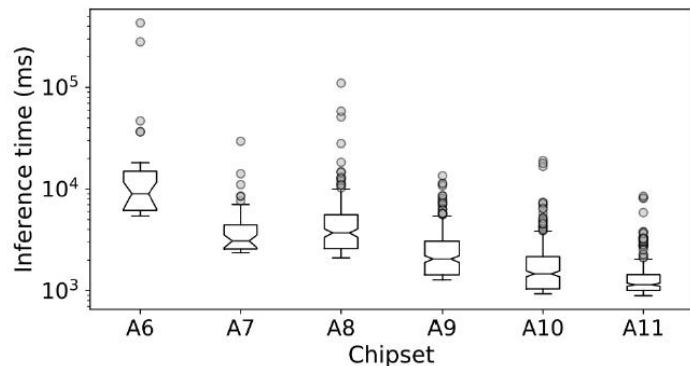


Figure 10: The inference time performance improves over generations of iPhones. However, within each generation, significant inference performance variability is observed with a large number of outliers.

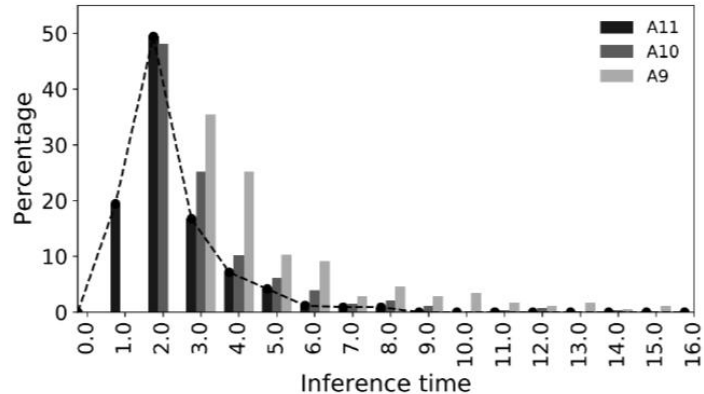


Figure 11: The inference time follows an approximate Gaussian distribution with the mean centered at 2.02ms and the standard deviation of 1.92ms.

/ Interference

- Contention for shared resources also applies to accelerators, especially in a datacenter environment.

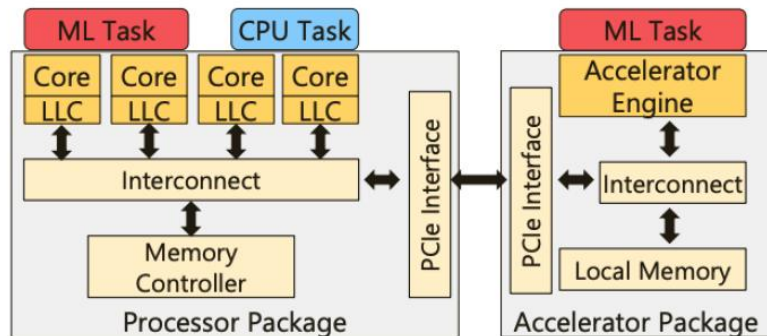


Figure 4: Architecture of an accelerated platform.

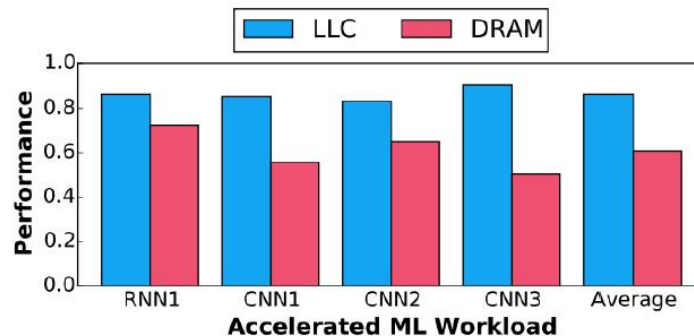
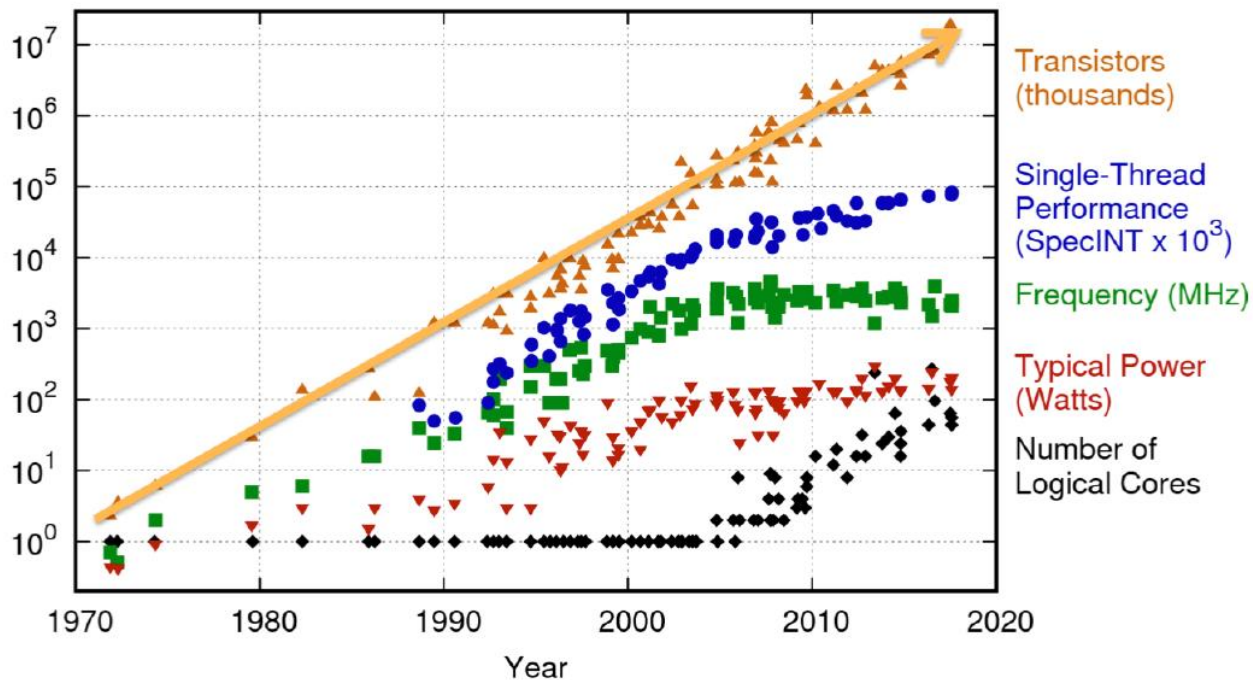


Figure 5: Workload sensitivity to shared resource interference. Performance is normalized to no interference.

Outline

- **End-to-End Deployment**
 - **AI Processing Pipeline**
 - **Practical Constraints:**
Thermal, Performance Variability, Interference
- **Accelerator-Level Parallelism (ALP)**
 - **X-level Parallelism**
 - **ALP in Mobile SoC**
 - **Open Challenges**

/Advances in Processor Performance



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten.
New plot and data for 2010-2017 by K. Rupp from <https://www.karlrupp.net/wp-content/uploads/2018/02/42-years-processor-trend.png>

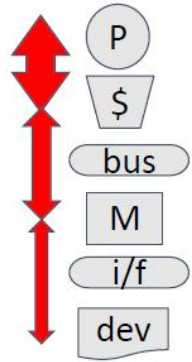
/Great Ideas in Computer Architecture

- Abstractions
- Moore's Law
- Principles of Locality/Memory Hierarchy

•Parallelism

- Performance Measurement & Improvement
- Dependability via Redundancy

/X-Level Parallelism in Computer Architecture

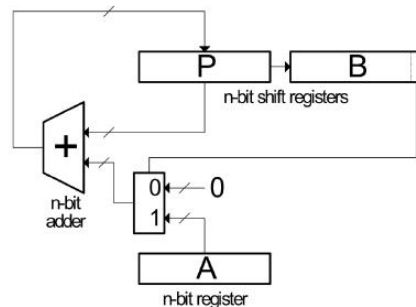


1 CPU

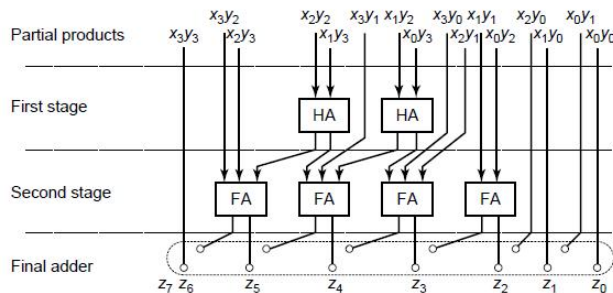
BLP+ILP
Bit/Instrn-Level
Parallelism

/Bit-Level Parallelism (BLP)

- Early computers with few transistors
 - Compute a result in many steps
 - E.g., 1 multiplication partial sum per cycle for “shift-and-add” multiplier
- Bit-Level Parallelism
 - More more bits in parallel
 - Larger words improve BLP:
 - 8b->16b->32b->64b
 - Recent trends:
 - Smaller word size preferred for better efficiency, e.g., DNN inference HW.



- “Shift-and-add” multiplier
- Performance: N cycles for N-bit additions



- Parallel Wallace Tree Multiplier

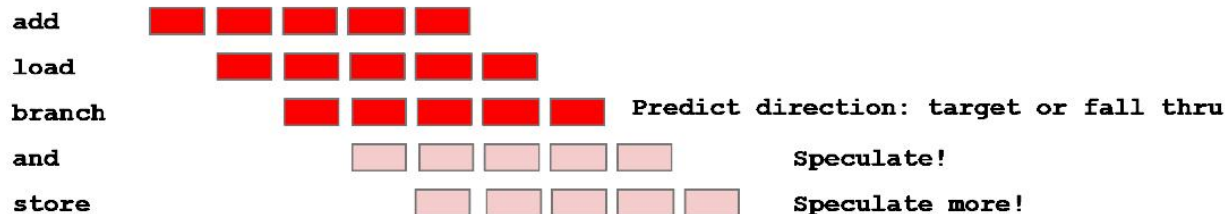
Mark Hill, Accelerator-Level Parallelism, Presentation, 2020

/ Instruction-Level Parallelism (ILP)

- Processors logically do instructions sequentially (time →)

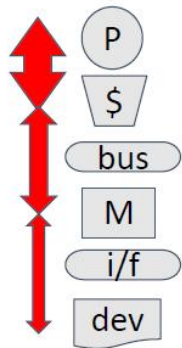


- Though they actually execute instructions in parallel, i.e., instruction-level parallelism.



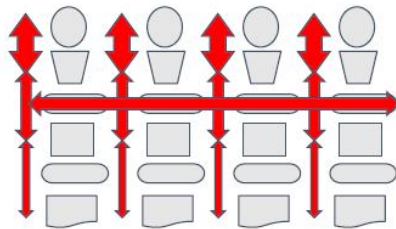
- Completely hides hardware complexity from software.

/X-Level Parallelism in Computer Architecture



1 CPU

BLP+ILP
Bit/Instrn-Level
Parallelism



Multiprocessor

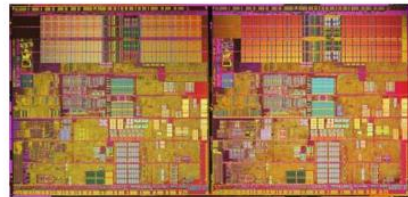
+ TLP
Thread-Level
Parallelism

/ Thread-Level Parallelism (TLP)

- HW: Multiple sequential processors
- SW: Each runs asynchronous thread
- SW must partition work, synchronize, & manage communication
 - E.g., pThreads, OpenMP, MPI
- Less easy for software but:
 - Bifurcation: experts program TLP; others use it.

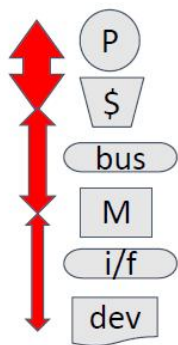


CDC 6600, 1964,
(TLP via multithreaded processor)

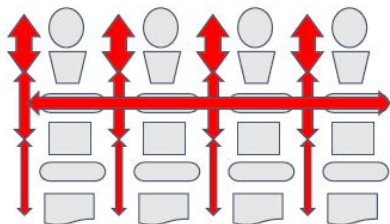


Intel Pentium Pro Extreme Edition,
early 2000s

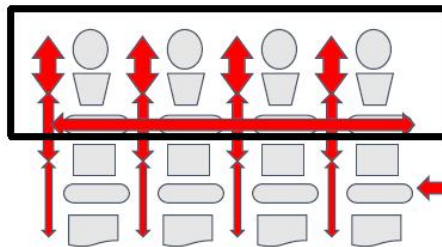
/X-Level Parallelism in Computer Architecture



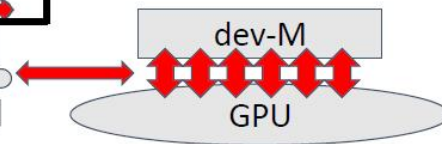
1 CPU
BLP+ILP
Bit/Instrn-Level
Parallelism



Multiprocessor
+ TLP
Thread-Level
Parallelism



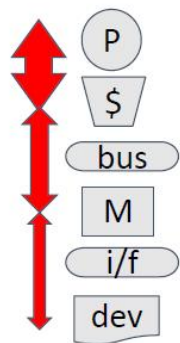
Multicore
+ TLP
Thread-Level
Parallelism



+ Discrete GPU
+ DLP
Data-Level
Parallelism

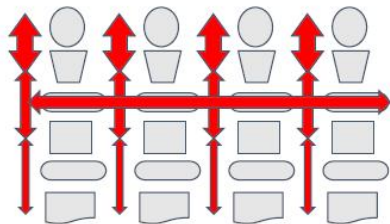
Mark Hill, Accelerator-Level Parallelism, Presentation, 2020

/X-Level Parallelism in Computer Architecture



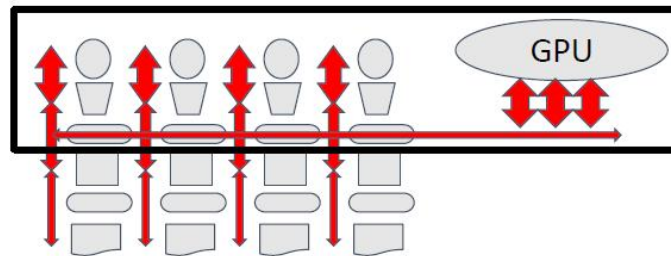
1 CPU

BLP+ILP
Bit/Instrn-Level
Parallelism



Multiprocessor

+ TLP
Thread-Level
Parallelism



Multicore + Integrated GPU

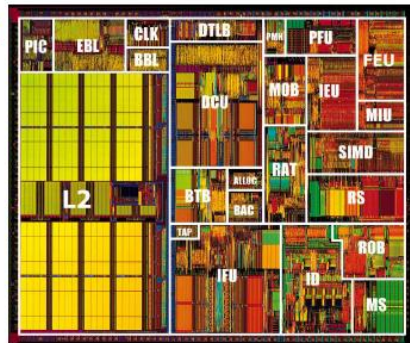
+ TLP
Thread-Level
Parallelism

+ DLP
Data-Level
Parallelism

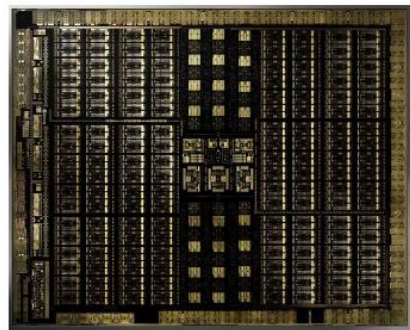
Mark Hill, Accelerator-Level Parallelism, Presentation, 2020

/Data-Level Parallelism

- Perform the same operation on different data elements
 - Single-Instruction-Multiple-Data (SIMD)
- DLP success: General-Purpose GPUs
 - Single-Instruction-Multiple-Threads (SIMT)
 - SW (CUDA) & Libraries (math & ML)
 - Bifurcation again:
 - Experts program SIMT (TLP + DLP);
 - Others use it.



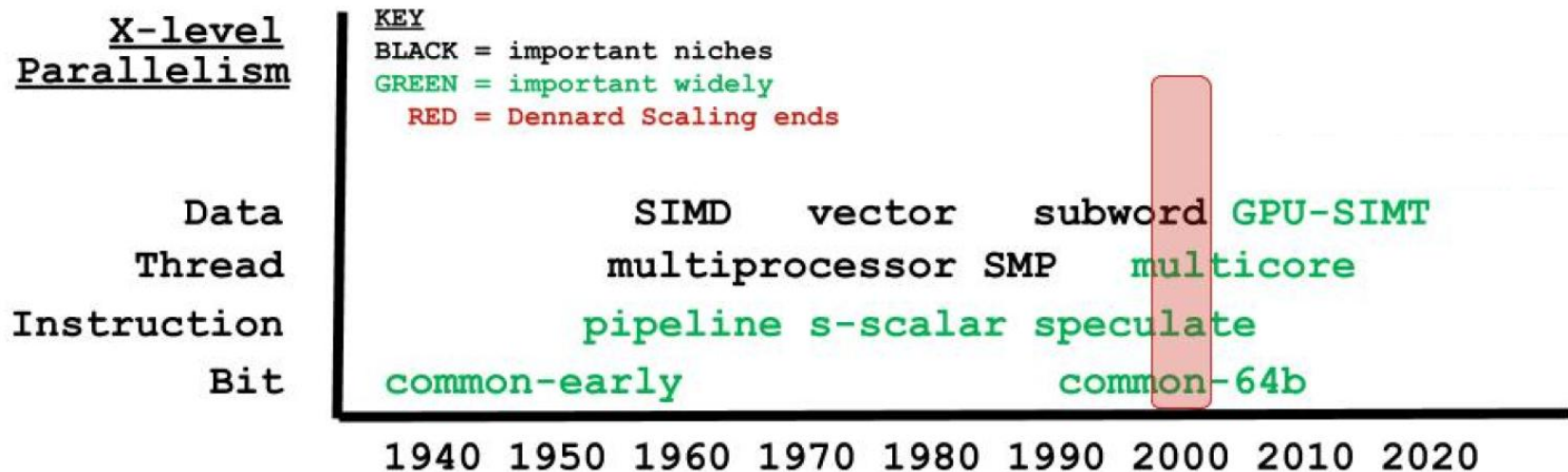
Intel Pentium III



NVIDIA Turing

/X-Level Parallelism in Computer Architecture

X-level Parallelism



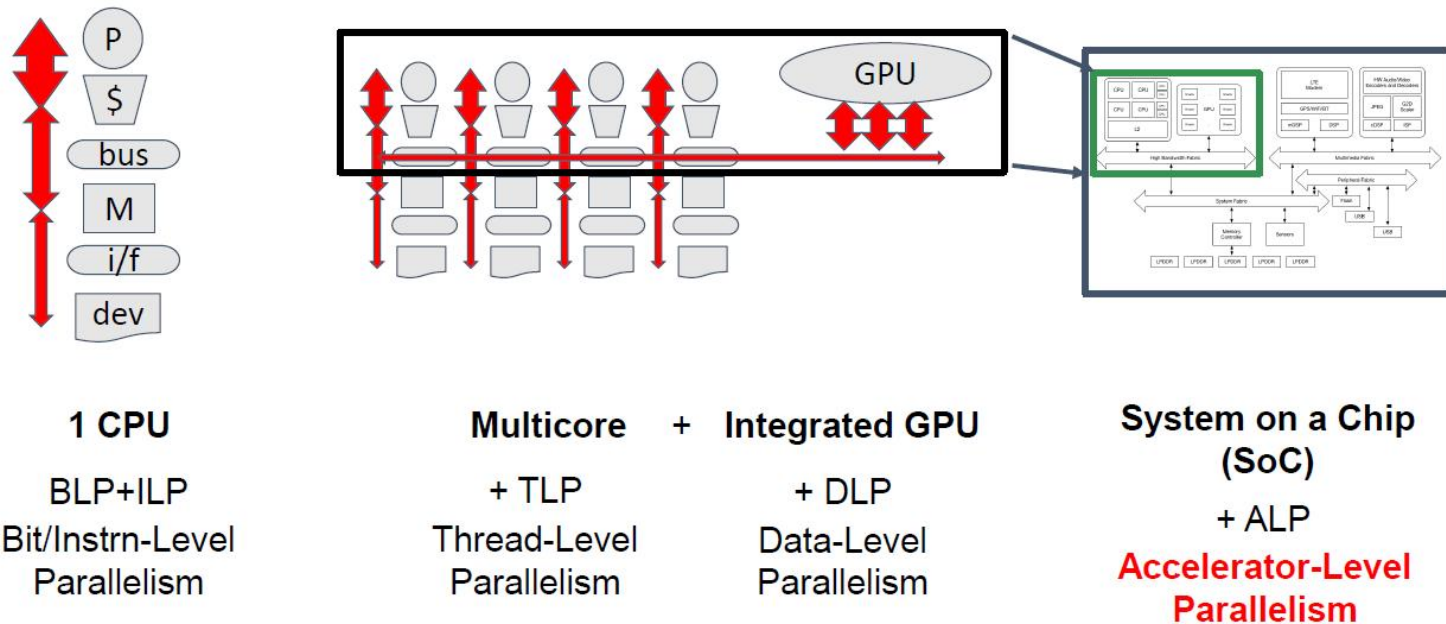
Mark Hill and Vijay Janapa Reddi, Accelerator-Level Parallelism, arXiv'2019

/Outline

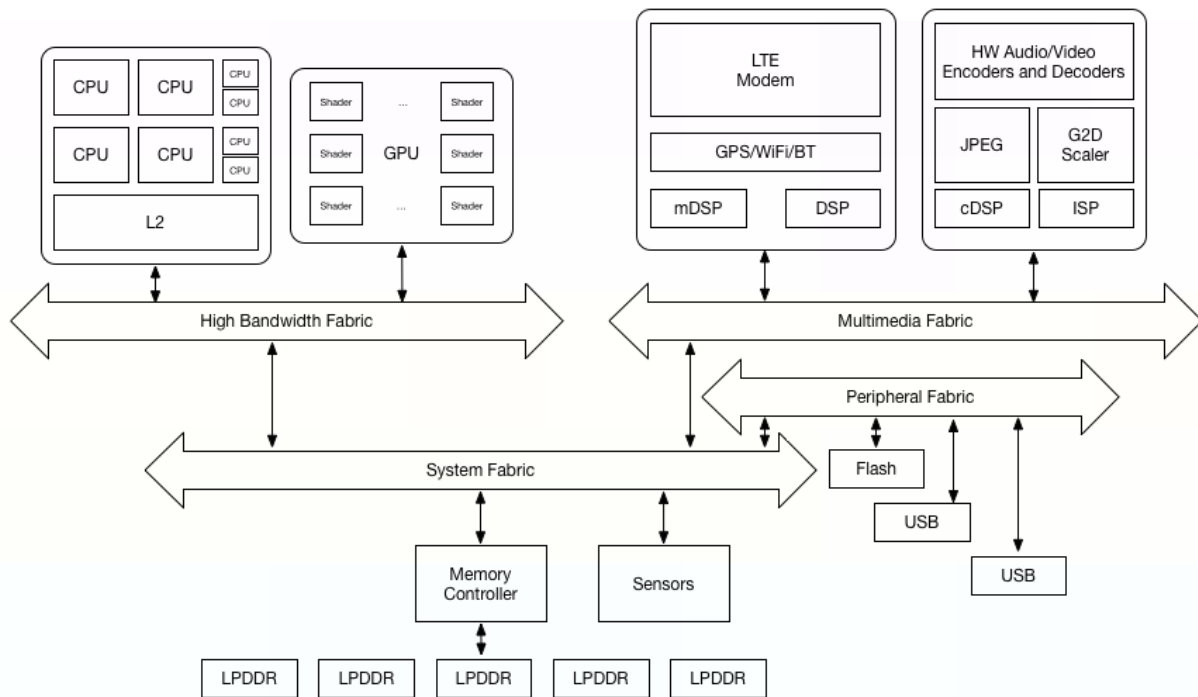
- **End-to-End Deployment**
 - **AI Processing Pipeline**
 - **Practical Constraints:**
Thermal, Performance Variability, Interference
- **Accelerator-Level Parallelism (ALP)**
 - **X-level Parallelism**
 - **ALP in Mobile SoC**
 - **Open Challenges**

/X-Level Parallelism in Computer Architecture

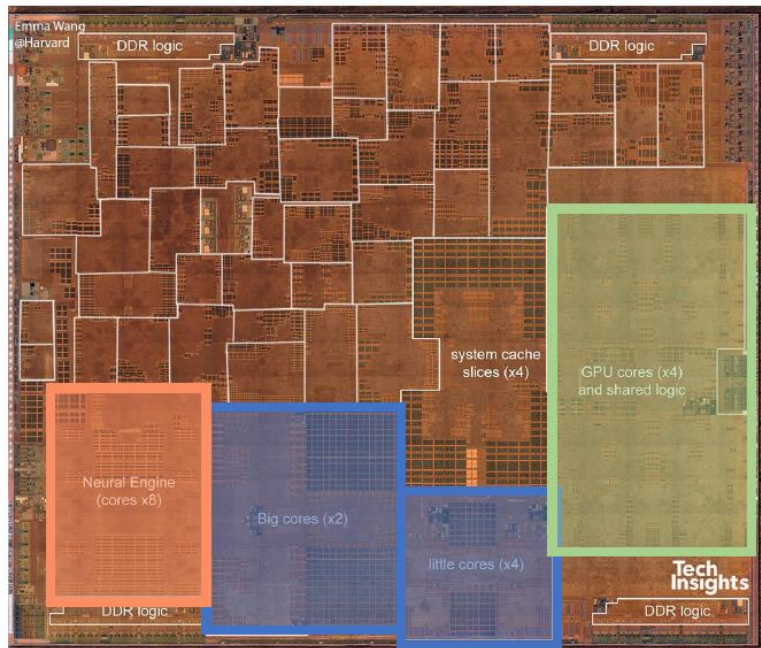
- Accelerator-level parallelism (ALP): the parallelism of different workload components concurrently executing on multiple accelerators.



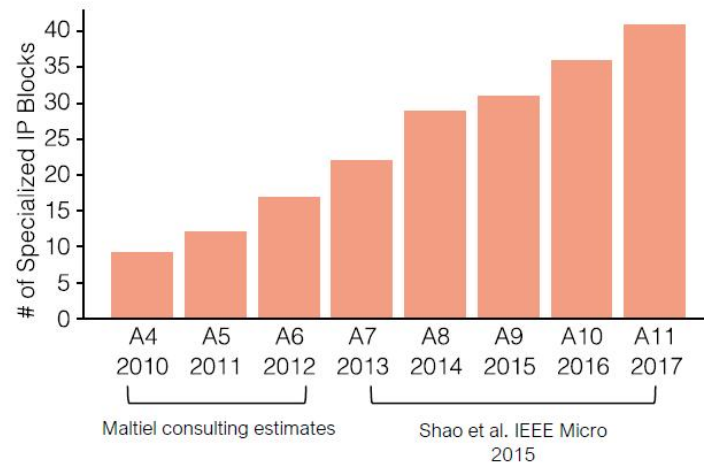
/ Mobile SoC HW



/CPU, GPU, and Accelerators



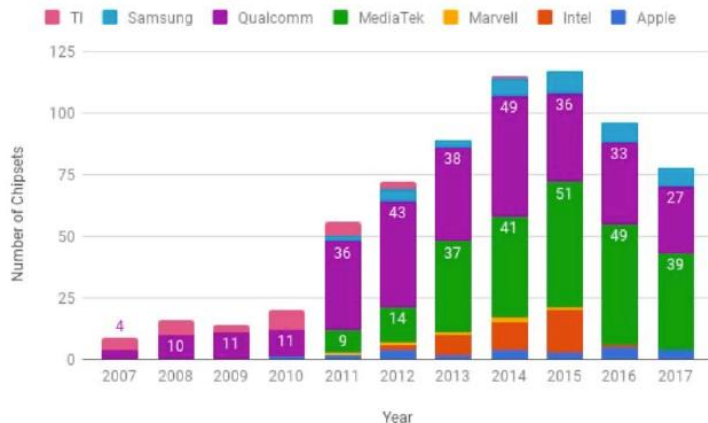
Apple A12 SoC



<http://vlsiarch.eecs.harvard.edu/research/accelerators/die-photo-analysis/>

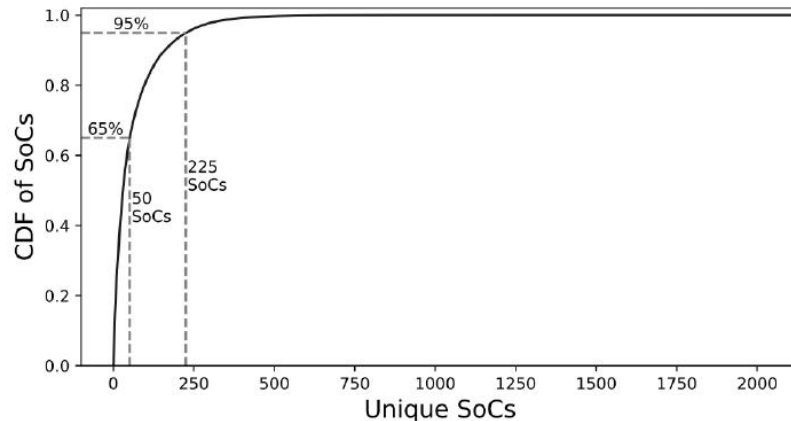
/ Mobile SoC Diversity

- 10s-100s of SoCs introduced to the market every years across different SoC vendors.



Two Billion Devices and Counting: An Industry Perspective on the State of Mobile Computer Architecture, IEEE Micro'2018

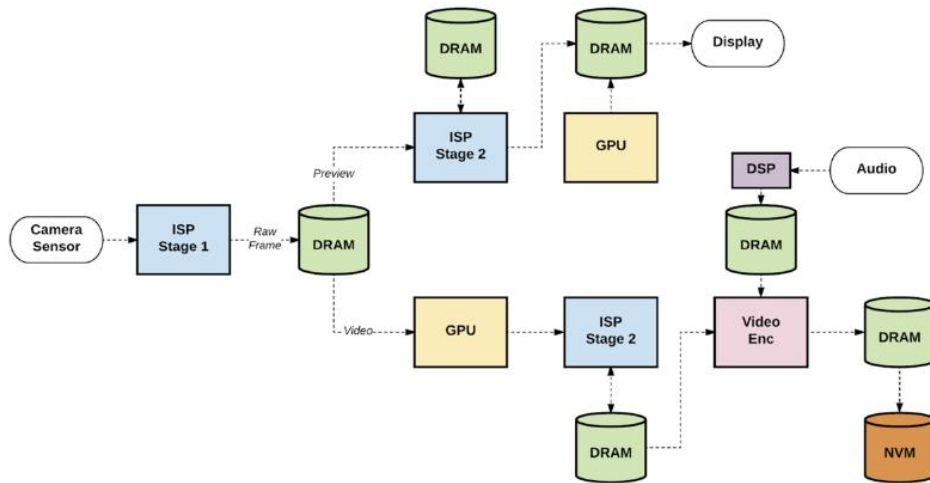
- No standard mobile SoC to optimize for. Top 50 most common SoCs -> only 65% of the market.



Machine Learning at Facebook: Understanding Inference at the Edge, HPCA'2019

/ Mobile SoC Usecase

- Mainstream architecture has long focused on general-purpose CPUs and GPUs.
- In an SoC, multiple IP blocks are active at the same time and communicate frequently with each other.
- Example:
 - Recording a 4K video
 - Camera -> ISP
 - “Preview stream” for display
 - “Video stream” for storage
 - DRAM for data sharing



Two Billion Devices and Counting: An Industry Perspective on the State of Mobile Computer Architecture, IEEE Micro'2018

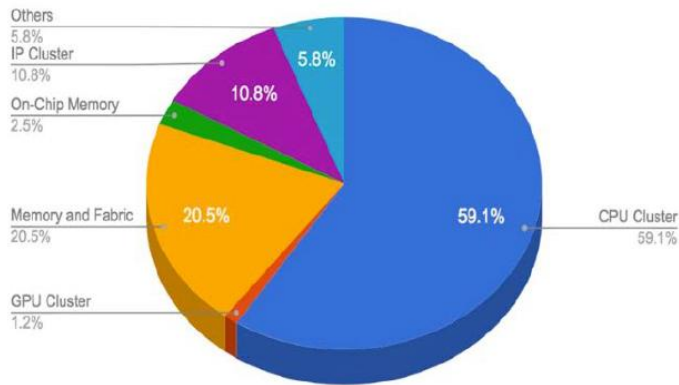
/ Mobile SoC Usecase

- Multiple accelerators are running concurrently for different usecases.

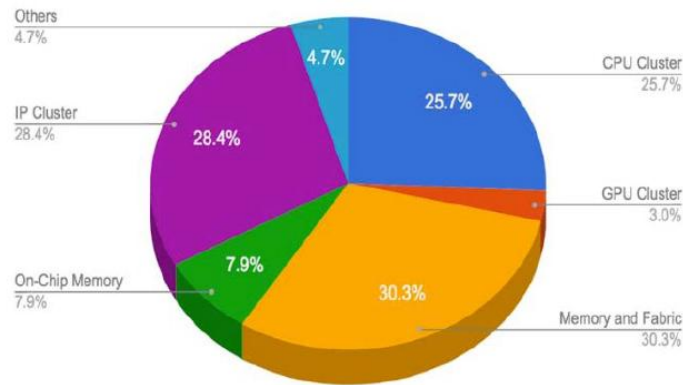
Accelerators (IPs) → Usecases (rows)	CPUs (AP)	Display	Media Scaler	GPU	Image Signal Proc.	JPEG	Pixel Visual Core	Video Decoder	Video Encoder	Dozens More
Photo Enhancing	X	X		X	X	X	X			
Video Capture	X	X		X	X				X	
Video Capture HDR	X	X		X	X				X	
Video Playback	X	X	X	X				X		
Image Recognition	X	X	X	X						

/ Mobile SoC Usecase

- Different usecases have different power profiles.



Web browser scrolling
(total 651.77 mW)



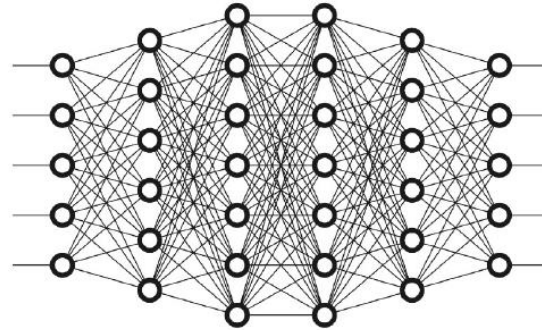
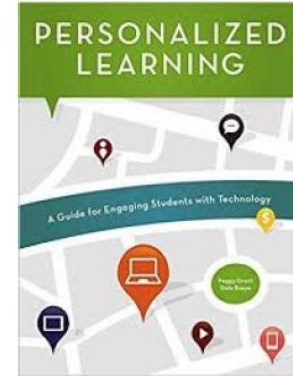
4K video recording
(total 3,473.55mW)

Two Billion Devices and Counting: An Industry Perspective on the State of Mobile Computer Architecture, IEEE Micro'2018

/Outline

- **End-to-End Deployment**
 - **AI Processing Pipeline**
 - **Practical Constraints:**
Thermal, Performance Variability, Interference
- **Accelerator-Level Parallelism (ALP)**
 - **X-level Parallelism**
 - **ALP in Mobile SoC**
 - **Open Challenges**

/Future Apps Demand Much More Computing

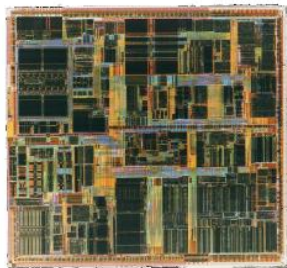


Mark Hill, Accelerator-
Level Parallelism,
Presentation, 2020

/X-level Parallel Hardware + Software (Model)

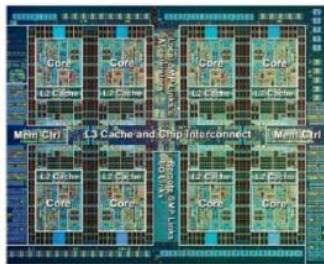
Single-threaded
languages,
compilers,
runtimes, etc.

hides parallelism



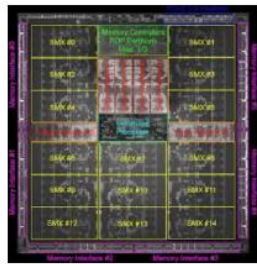
Intel Pentium Pro
BLP+ILP

Software
abstracts TLP,
e.g., pThreads,
OpenMP, & MPI.
Also **hidden** in
cloud, etc.



IBM Power 7
BLP+ILP+TLP

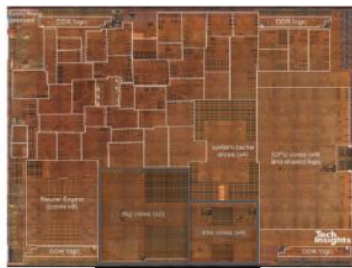
Software **abstracts**
TLP+DLP, e.g.,
CUDA, OpenCL, &
graphics OpenGL
Also **hidden** in
cloud, etc.



Nvidia GK110
BLP+TLP+DLP

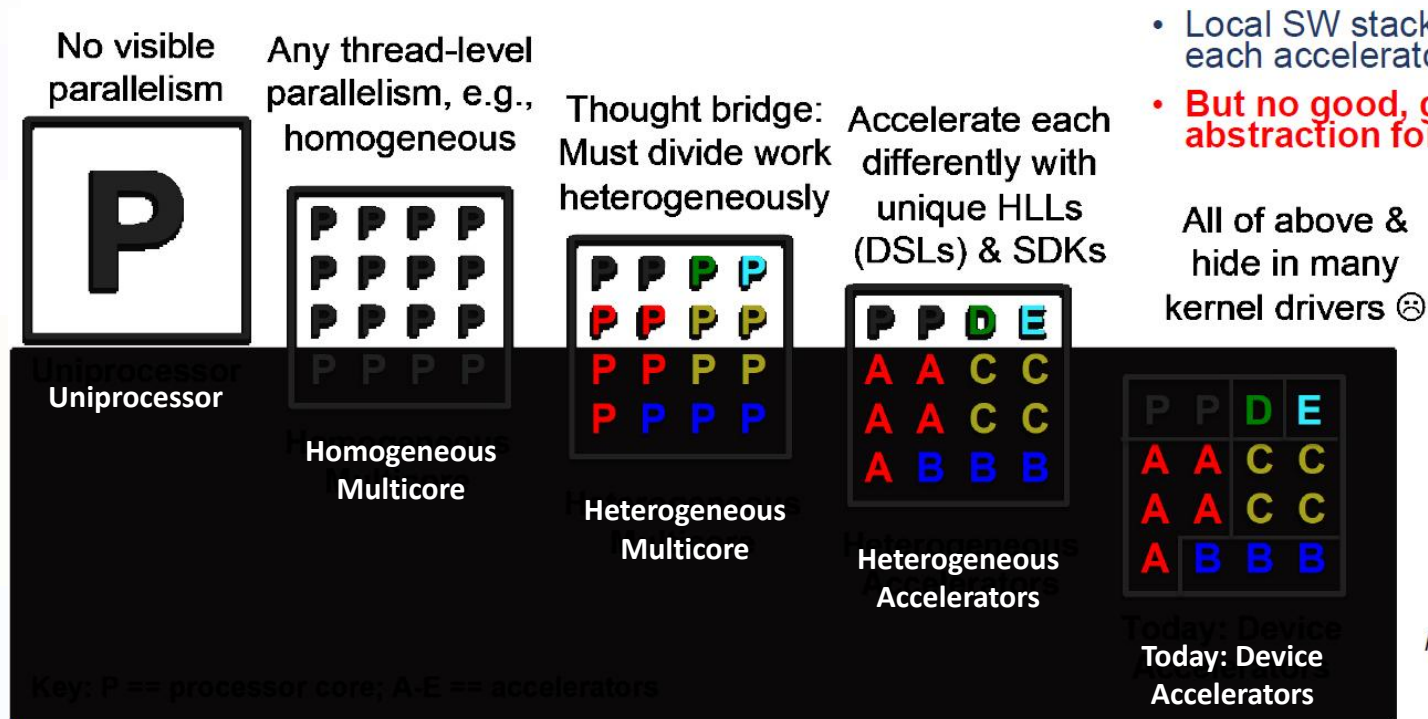
Local software stack
abstracts each
accelerator.

**But no good, general
software abstraction
for SoC ALP!**



Apple A12
BLP+ILP+TLP+DLP+ALP

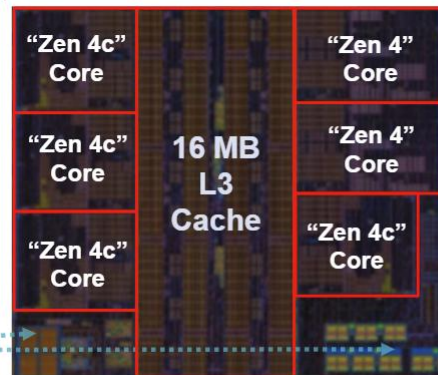
/ALP Challenges & Opportunities



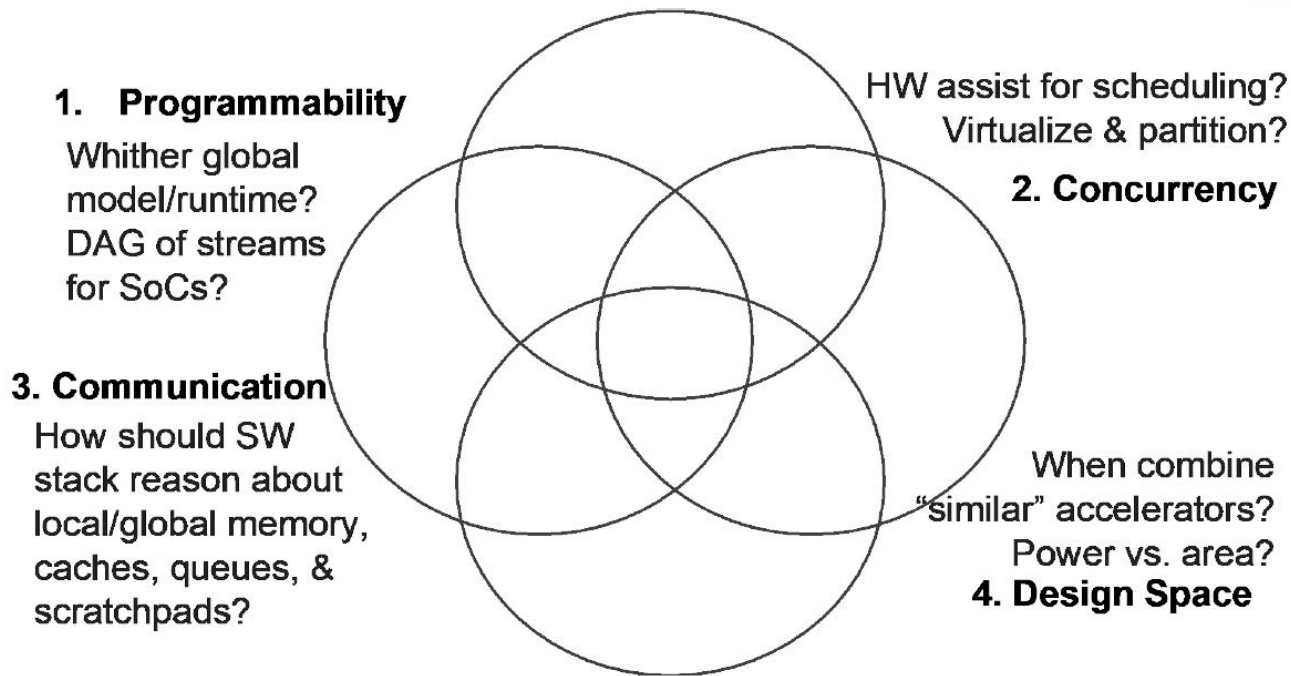
Mark Hill, Accelerator-Level Parallelism, Presentation, 2020

Heterogenous CCX for Client

- Eight core “Zen 4” APU (“Phoenix”) modified
 - Supports Heterogenous core design with 2 “Zen 4” cores + 4 “Zen 4c” cores
 - Delivers max performance when running few threads
 - Delivers improved many-threads performance
 - Has high-frequency 16 MB L3 cache to operate at “Zen 4” frequencies
- Like server configurations, “Zen 4c” delivers higher performance when operating at lower power per core
 - Delivers higher 6-core performance for TDPs $\leq 15W$
- Recovered area used to shrink the SoC



/ALP Challenges & Opportunities



/Review

- **End-to-End Deployment**
 - **AI Processing Pipeline: pre/post ML processing**
 - **Practical Constraints: end-to-end consideration**
Thermal, Performance Variability, Interference
- **Accelerator-Level Parallelism (ALP)**
 - **X-level Parallelism: BLP/ILP/TLP/DLP**
 - **ALP in Mobile SoC: diversity**
 - **Open Challenges:**
programmability, concurrency, communication, DSE